

RAPPORT DE PROJET DÉVELOPPEMENT BACKEND - DJANGO

Sujet: Plateforme de Gestion de
Bibliothèque de Films

Filière développement frontend
& Backend

Realiser par:
<>Yassine Bouasria

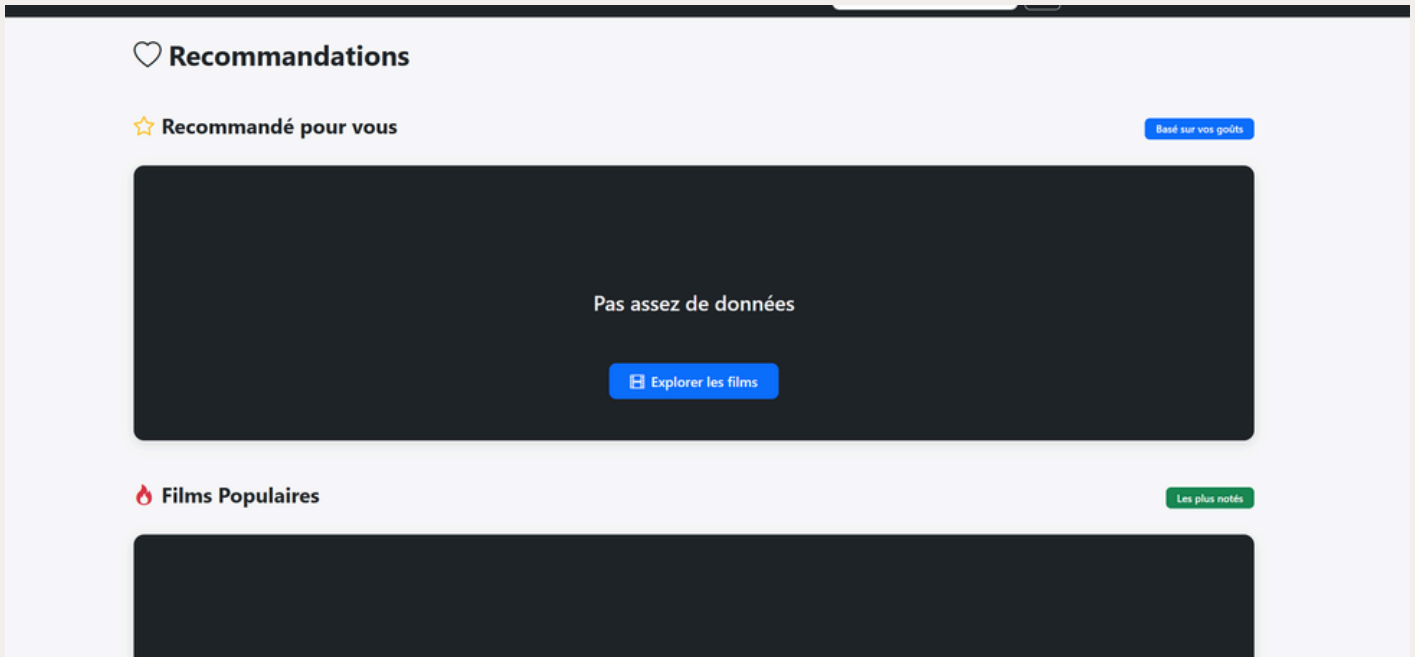
1.1 PRESENTATION DU PROJET

La plateforme "Filmo" est une application web de gestion de bibliotheque de films concue pour repondre aux besoins des cinephiles modernes. Developpee avec Django, elle combine robustesse technique et simplicité d'utilisation.

Ce projet s'articule autour de trois piliers fondamentaux :

1. Centralisation : Un catalogue riche et structure pour repertorier les oeuvres cinematographiques.
2. Interaction : Un systeme social permettant aux utilisateurs de s'exprimer via des avis et des notes.
3. Intelligence: Un moteur de recommandation qui apprend des gouts de l'utilisateur pour suggerer du contenu pertinent.

Ce rapport technique detaille l'implementation de chaque composant, en expliquant les choix architecturaux et en analysant les extraits de code les plus significatifs.



1.2 OBJECTIFS DE CE RAPPORT

Ce document a pour but de fournir une visibilité totale sur le fonctionnement interne de l'application, en liant les concepts de conception aux implementations techniques réelles.

2.1 LE PATTERN MTV

L'architecture du projet repose sur le pattern MTV (Model-Template-View), une variante du MVC classique adaptée au web. Ce choix garantit une maintenance facilitée et une scalabilité horizontale du projet.

MODELES (M)

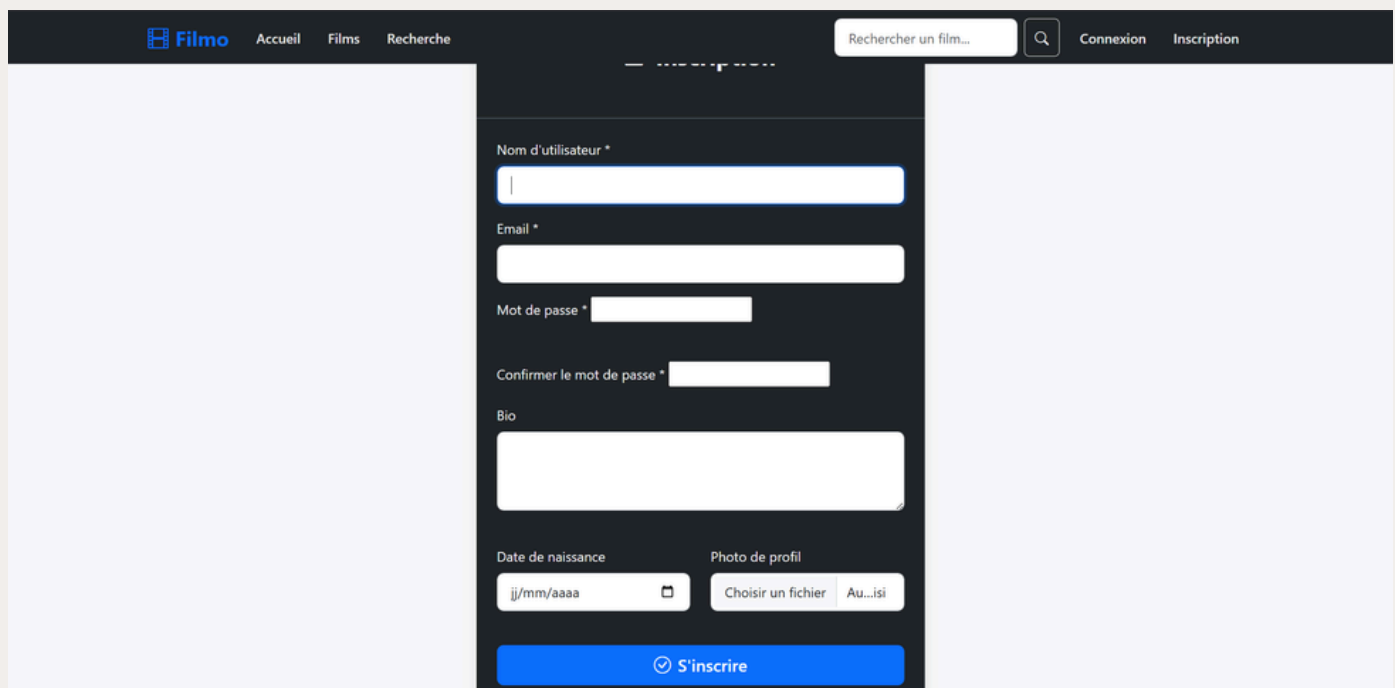
Ils definissent la structure des donnees et les contraintes d'integrite.

TEMPLATES (T)

La couche de presentation utilisant Bootstrap 5 pour une reponse adaptative.

VUES (V)

Le cerveau de l'application qui fait le pont entre les donnees et l'utilisateur.



The screenshot displays a web application interface for a movie-related site. The top navigation bar is dark blue with the 'Filmo' logo and links for 'Accueil', 'Films', and 'Recherche'. A search bar with the placeholder 'Rechercher un film...' and a magnifying glass icon is on the right, along with links for 'Connexion' and 'Inscription'. The main content area features a registration form with the following fields: 'Nom d'utilisateur *' (text input), 'Email *' (text input), 'Mot de passe *' (password input), 'Confirmer le mot de passe *' (password input), and 'Bio' (text area). Below these are 'Date de naissance' (date picker) and 'Photo de profil' (file upload button labeled 'Choisir un fichier' and 'Au...isi'). A blue 'S'inscrire' button with a checkmark icon is at the bottom of the form.

3.1 ABSTRACTION ET ORM

Le coeur de Filmo reside dans son schema de donnees. L'utilisation d'un ORM (Object-Relational Mapper) permet de manipuler la base de donnees SQLite comme des objets Python, assurant une abstraction totale de la couche SQL.

```
class User(AbstractUser):
    """Custom user model with role-based permissions"""

    class Role(models.TextChoices):
        SPECTATOR = 'SPECTATOR', _('Spectateur')
        ADMIN = 'ADMIN', _('Administrateur')

    base_role = Role.SPECTATOR

    role = models.CharField(
        max_length=50,
        choices=Role.choices,
        default=Role.SPECTATOR,
    )

    profile_picture = models.ImageField(
        upload_to='profiles/',
        null=True,
        blank=True,
        default='profiles/default.jpg'
    )

    bio = models.TextField(max_length=500, blank=True)
    favorite_genres = models.ManyToManyField('Genre', blank=True, related_name='favorited_by')
    birth_date = models.DateField(null=True, blank=True)

    def save(self, *args, **kwargs):
        if self.role == self.Role.ADMIN:
            self.is_staff = True
            self.is_superuser = True
        else:
            self.is_staff = False
            self.is_superuser = False
        super().save(*args, **kwargs)
```

L'explication du code ci-dessus : La methode `save()` est surchargee pour synchroniser les drapeaux Django standards (`is_staff`) avec notre systeme de roles personnalise.

4.1 STRUCTURE DU MODELE FILM

Le modele 'Film' stocke les metadonnees essentielles. L'implementation de proprietes dynamiques comme 'average_rating' permet de calculer en temps reel la note moyenne sans stocker de donnees redondantes en base.

The screenshot shows the login interface of a website called 'Filmo'. The header is dark blue with the 'Filmo' logo and navigation links: 'Accueil', 'Films', and 'Recherche'. A search bar with the placeholder 'Rechercher un film...' and a magnifying glass icon is on the right. The main content area features a dark blue login card with the title 'Connexion' and a key icon. It contains two input fields for 'Nom d'utilisateur' and 'Mot de passe', a checkbox for 'Se souvenir de moi', and a blue 'Se connecter' button. A link for 'Inscrivez-vous' is at the bottom of the card. The footer is dark blue with the 'Filmo' logo and copyright text: '© 2024 Filmo. Tous droits réservés.'

4.2 CATEGORISATION PAR GENRES

La relation ForeignKey entre Film et Genre permet un filtrage efficace et une organisation logique du catalogue.

5.1 TRAITEMENT DES REQUETES

Les vues pilotent la logique de navigation. Elles traitent les requetes HTTP, interagissent avec les modeles et renvoient les templates appropriées

5.2 LE MOTEUR DE RECHERCHE

Le moteur de recherche utilise l'objet 'Q' pour effectuer des requetes 'OR' complexes. Cela permet a l'utilisateur de trouver un film en saisissant une partie du titre, du nom du realisateur ou meme de la description.

```
def search_films(request):
    """Search films by title, genre, or date"""
    query = request.GET.get('q', '')
    genre_id = request.GET.get('genre')
    year = request.GET.get('year')
    films = Film.objects.all()

    if query:
        films = films.filter(
            Q(title__icontains=query) |
            Q(description__icontains=query) |
            Q(director__icontains=query)
        )
```

Administration de Django

BIENVENUE, ADMIN. VOIR LE SITE / MODIFIER LE MOT DE PASSE / DÉCONNECTION

Site d'administration

AUTHENTIFICATION ET AUTORISATION

Groupes

+ Ajouter

Modification

MOVIES

Films

+ Ajouter

Modification

Genres

+ Ajouter

Modification

Reviews

+ Ajouter

Modification

Utilisateurs

+ Ajouter

Modification

Watchlists

+ Ajouter

Modification

Actions récentes

Mes actions

Mad Max: Fury Road

Film

The Prestige

Film

The Lord of the Rings

Film

Forrest Gump

Film

Pulp Fiction

Film

Spirited Away

Film

Whiplash

Film

The Shawshank Redemption

Film

Gladiator

Film

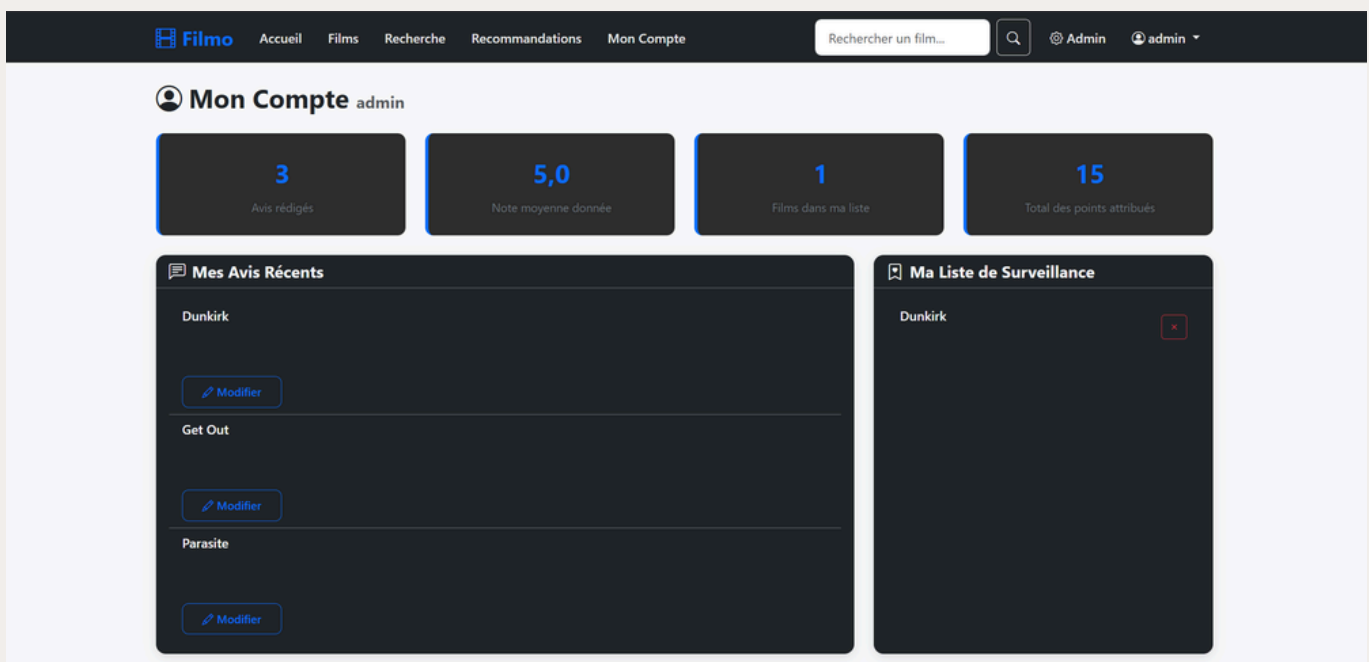
The Matrix

Film

localhost:8000/admin/

6.1 LE TABLEAU DE BORD (DASHBOARD)

Le dashboard centralise l'activité de l'utilisateur. Il affiche non seulement ses avis mais aussi ses statistiques personnelles d'engagement.



6.2 LA LISTE DE SURVEILLANCE (WATCHLIST)

La watchlist utilise une relation Many-to-Many via un modèle intermédiaire pour enregistrer la date d'ajout du film.

7.1 PRINCIPE DU FILTRAGE PAR CONTENU

L'algorithme de recommandation est basé sur le filtrage par contenu. Il analyse les genres des films que l'utilisateur a notés positivement (≥ 4) et extrait les films de ces mêmes genres qu'il n'a pas encore vus.

Administration de Django

BIENVENUE, ADMIN. VOIR LE SITE / MODIFIER LE MOT DE PASSE / DÉCONNEXION

Accueil • Movies • Films

Écrivez ici pour filtrer...

AUTHENTIFICATION ET AUTORISATION

GROUPE + Ajouter

MOVIES

Films + Ajouter

Genres + Ajouter

Reviews + Ajouter

Utilisateurs + Ajouter

Watchlists + Ajouter

Sélectionnez l'objet film à changer

Rechercher

Action : [dropdown] Exécuter 0 sur 21 sélectionné

TITLE	GENRE	RELEASE DATE	DIRECTOR	NOTE MOYENNE	AVIS	AFFICHE
☐ Parasite	Thriller	30 mai 2019	Bong Joon Ho	5.0/5	1 avis	
☐ Dunkirk	War	21 juillet 2017	Christopher Nolan	5.0/5	1 avis	
☐ Get Out	Horror	24 février 2017	Jordan Peele	5.0/5	1 avis	
☐ La La Land	Romance	9 décembre 2016	Damien Chazelle	Non noté	0 avis	
☐ Arrival	Science Fiction	11 novembre 2016	Denis Villeneuve	Non noté	0 avis	
☐ Mad Max: Fury Road	Action	15 mai 2015	George Miller	Non noté	0 avis	

AJOUTER FILM +

FILTRE

Afficher les nombres

Par genre

- Tout
- Action
- Adventure
- Animation
- Comedy
- Crime
- Documentary
- Drama
- Fantasy
- Horror
- Mystery
- Romance
- Science Fiction
- Thriller
- War
- Western

Par release date

- Toutes les dates
- Aujourd'hui
- Les 7 derniers jours

L'explication : En identifiant les genres 'aimés' (notes ≥ 4), l'algorithme réduit l'espace de recherche aux films les plus susceptibles de plaire, améliorant ainsi la rétention utilisateur.

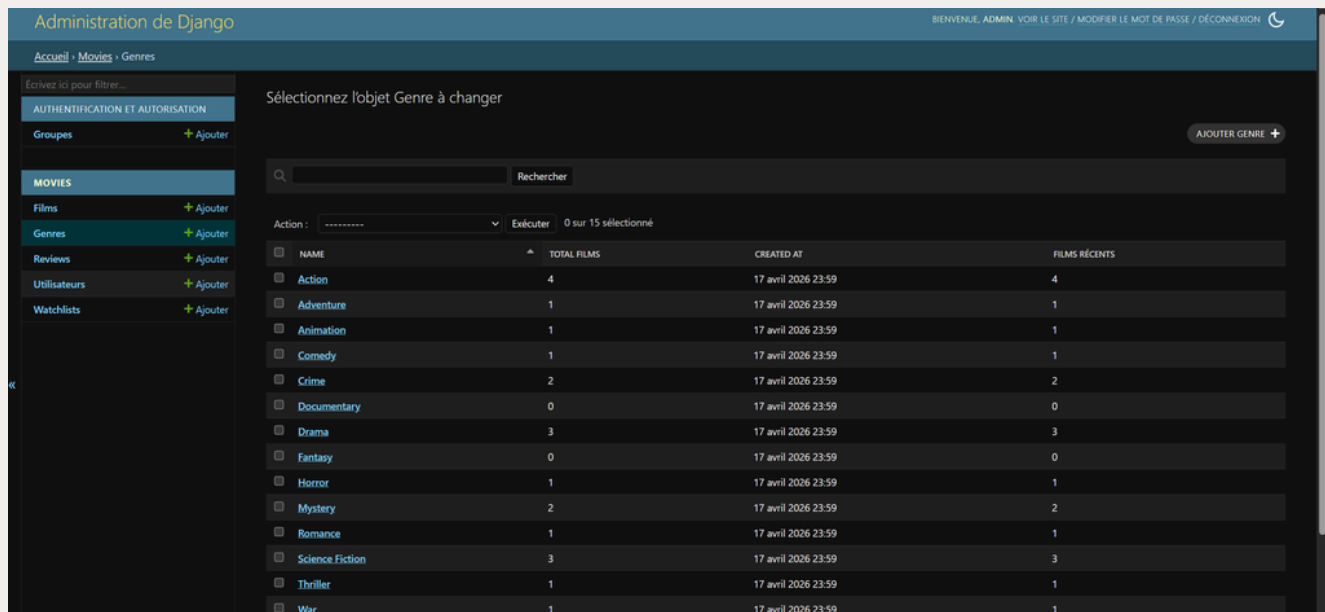
8.1 SIGNAUX ET GROUPES

La securite est renforcee par des signaux Django qui automatisent l'assignation des groupes de permissions lors de la creation d'un compte. Les administrateurs recoivent automatiquement les droits 'is_staff' et 'is_superuser'

```
@receiver(post_save, sender=User)
def assign_user_to_group(sender, instance, created, **kwargs):
    """Assign user to appropriate group based on role"""
    if created:
        if instance.role == User.Role.ADMIN:
            admin_group, _ = Group.objects.get_or_create(name='Administrators')
            instance.groups.add(admin_group)
        else:
            spectator_group, _ = Group.objects.get_or_create(name='Spectators')
            instance.groups.add(spectator_group)
```

8.2 PROTECTION DES VUES

L'utilisation systematique de @login_required et de tests de roles empeche tout acces non autorise aux fonctions de modification du catalogue



9.1 DJANGO REST FRAMEWORK

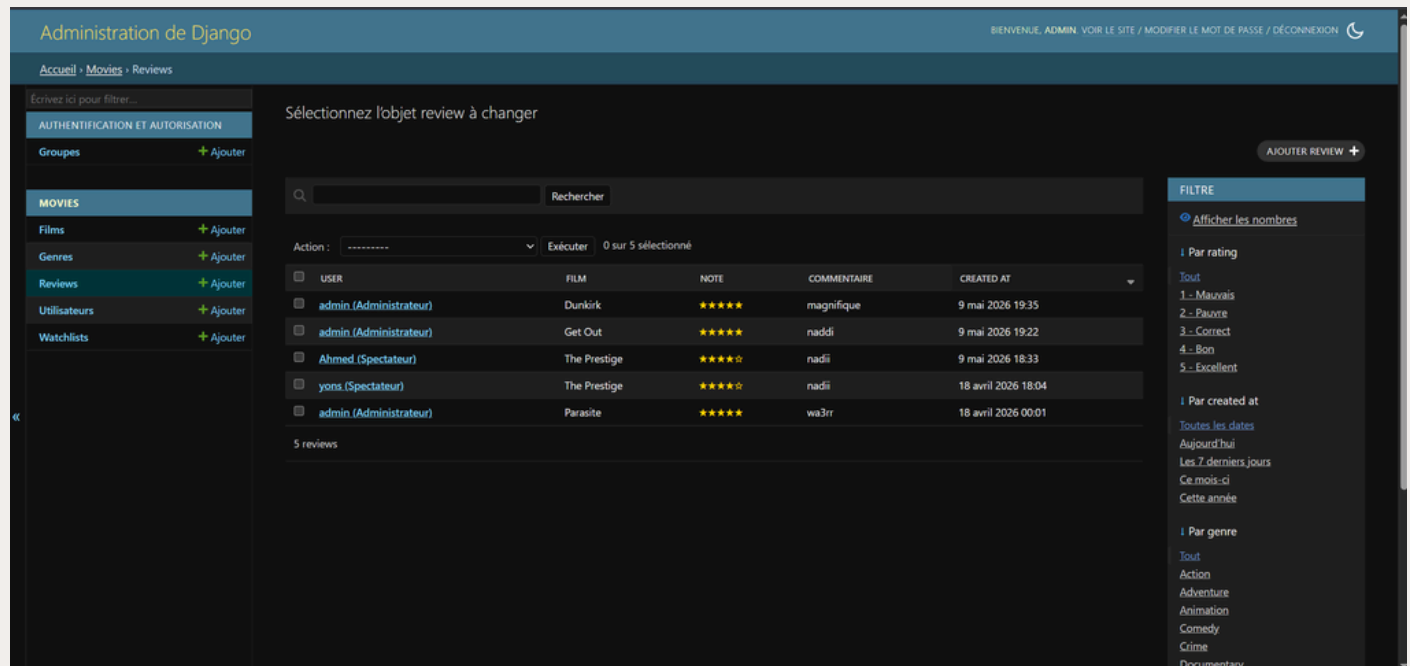
L'API permet d'exposer les données du projet au format JSON pour une utilisation par des applications mobiles ou des partenaires.

```
class FilmSerializer(serializers.ModelSerializer):
    """Serializer for Film model with computed fields"""
    genre = GenreSerializer(read_only=True)
    genre_id = serializers.PrimaryKeyRelatedField(
        queryset=Genre.objects.all(),
        write_only=True,
        source='genre',
        required=False,
        allow_null=True
    )
    average_rating = serializers.FloatField(read_only=True)
    review_count = serializers.IntegerField(read_only=True)
    is_in_watchlist = serializers.SerializerMethodField(read_only=True)

    class Meta:
        model = Film
        fields = [
            'id', 'title', 'genre', 'genre_id', 'description',
            'release_date', 'director', 'duration_minutes',
            'poster', 'trailer_url', 'average_rating', 'review_count',
            'is_in_watchlist', 'created_at', 'updated_at'
        ]
        read_only_fields = ['id', 'created_at', 'updated_at', 'average_rating', 'review_count', 'is_in_watchlist']
```

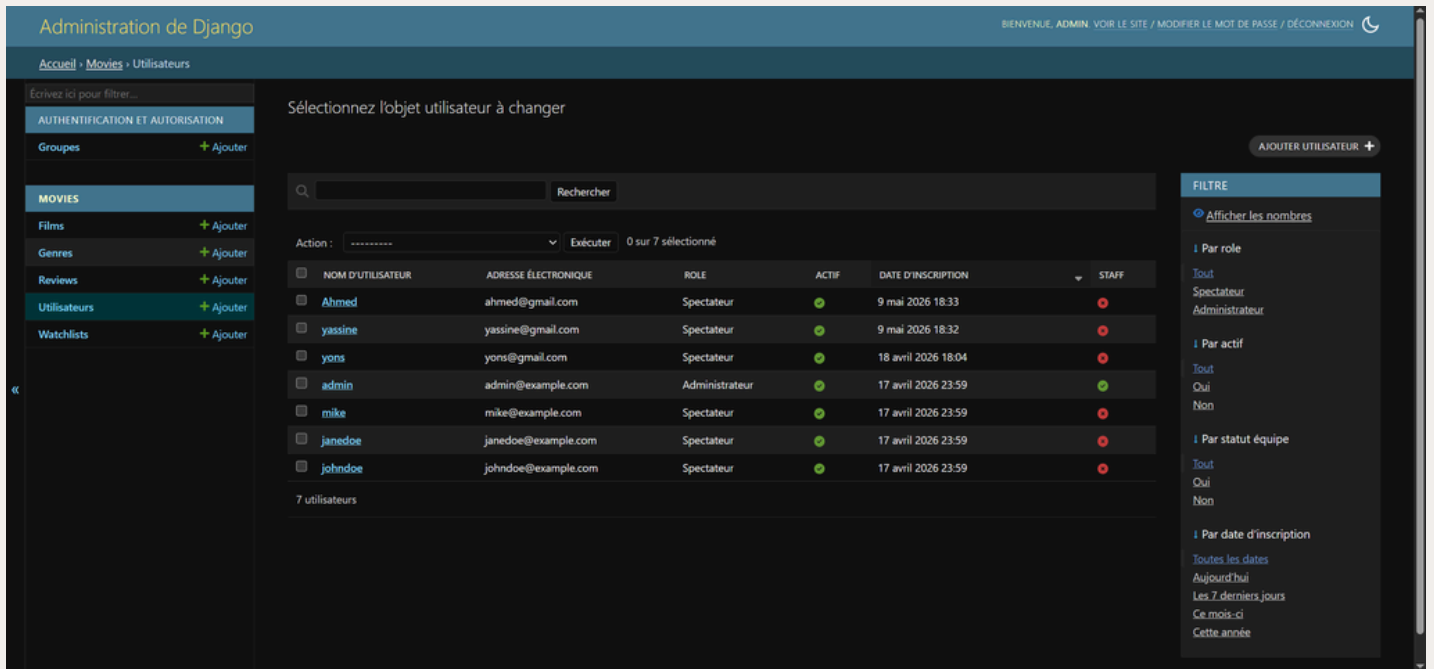
10.1 UTILISATION DE BOOTSTRAP 5

Le design est 'Mobile First'. L'interface s'adapte automatiquement a la taille de l'ecran (Smartphone, Tablette, Desktop).



10.2 FEEDBACK UTILISATEUR

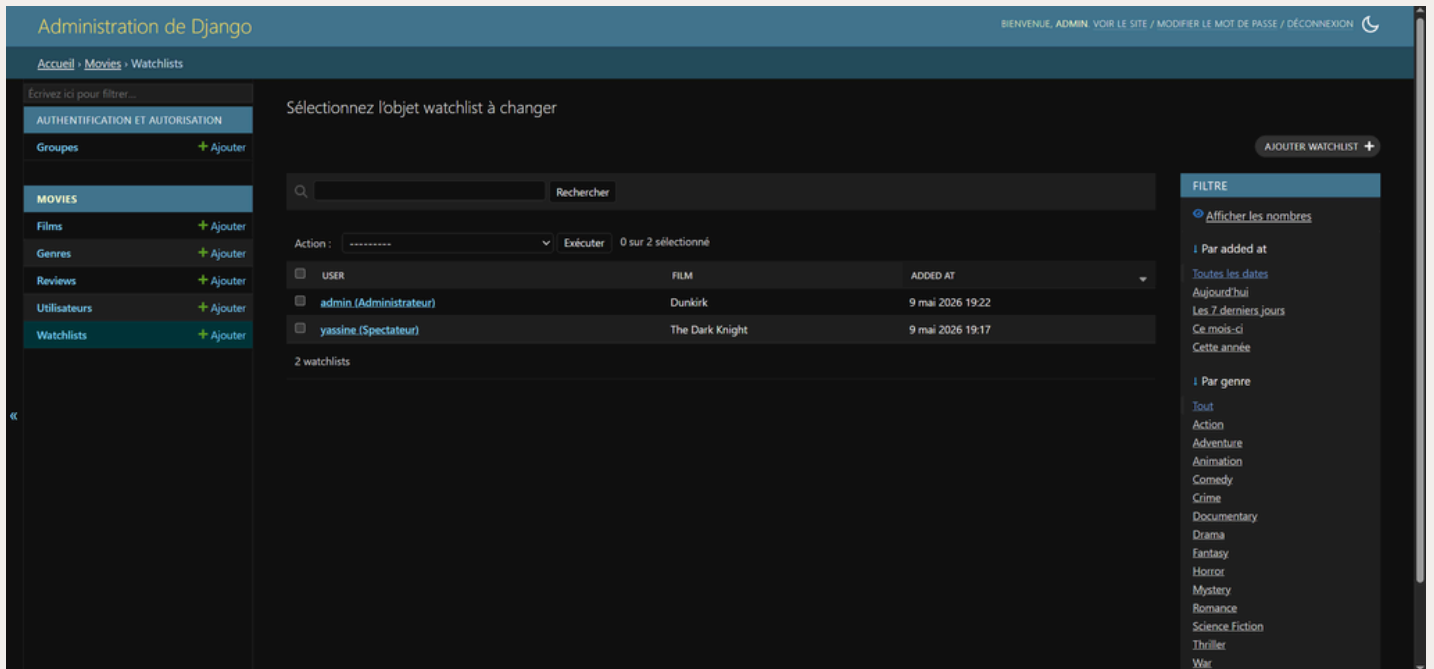
Le systeme utilise Django Messages pour confirmer chaque action (ajout d'avis, suppression de film, etc.) avec des alertes visuelles claires.



11.1 STRUCTURE DES DOSSIERS

Le projet est organisé de manière modulaire :

- movies/ : L'application principale contenant toute la logique métier.
- film_platform/ : Le noyau de configuration (settings, urls).
- templates/ : Les fichiers HTML structures par application.
- static/ & media/ : Les ressources visuelles et les fichiers uploadés.



Le projet Filmo est une base solide pour une plateforme cinématographique moderne. Son architecture propre et documentée permet d'envisager sereinement des évolutions majeures comme l'ajout de paiements (Stripe) ou l'intégration d'intelligence artificielle pour des recommandations encore plus fines.